

C++ · PERFORMANCE · MICROBENCHMARKS

Google Benchmark

Medindo desempenho de código C++ sem se enganar

Lucas Araújo

Setembro de 2026

```
$ ./benchmarks
Benchmark                                Time
-----
BM_LinearSearch/16                      8.3 ns
BM_LinearSearch/1024                    180 ns
BM_LinearSearch/65536                  10912 ns
BM_LinearSearch_BigO                    0.17 N

BM_BinarySearch/65536                   69.2 ns
BM_BinarySearch_BigO                     4.00 lgN
```

Por que não só std::chrono?

O jeito ingênuo de medir tempo tem armadilhas que não aparecem no número final

MEDIÇÃO MANUAL

```
auto t0 = std::chrono::steady_clock::now();

for (int i = 0; i < 1000; ++i)
    LinearContains(values, 42);

auto t1 = std::chrono::steady_clock::now();

std::cout << (t1 - t0) / 1000 << "\n";
// Mediu o quê, exatamente?
```

1

O compilador pode apagar o que você mede

O resultado não é usado; com -O3 o laço inteiro some.

2

Quantas iterações? 1000 é chute

Pouco demais vira ruído; demais desperdiça tempo.

3

Sem aquecimento

Cache, TLB e frequência da CPU ainda estão "frios".

4

Sem estatística

Uma execução só. Ficou mais rápido mesmo ou foi sorte?

O que é o Google Benchmark

Biblioteca open source do Google (licença Apache 2.0) para microbenchmarks em C++.

Você escreve só o trecho a medir. A biblioteca cuida do resto: quantas vezes rodar, como agregar e como exportar.

github.com/google/benchmark

Versão usada no piloto: v1.9.5

Iterações automáticas

Roda até o tempo estabilizar; você não escolhe o N de repetições.

Parâmetros e ranges

Um benchmark, vários tamanhos: Arg, Range, DenseRange.

Big-O automático

Ajusta $O(1)$, $O(N)$, $O(N \log N)$... aos dados e reporta o erro.

Fixtures e templates

Setup compartilhado e o mesmo teste para vários tipos.

Counters

items/s, bytes/s ou qualquer métrica sua como coluna.

JSON/CSV + compare.py

Exporta resultados e compara A/B com teste estatístico.

Setup com CMake

Sem instalar nada no sistema: o CMake baixa e compila junto com o projeto

CMakeLists.txt

```
INCLUDE(FetchContent)
SET(BENCHMARK_ENABLE_TESTING OFF CACHE BOOL "" FORCE)

FetchContent_Declare(
  googlebenchmark
  GIT_REPOSITORY https://github.com/google/benchmark.git
  GIT_TAG v1.9.5
  GIT_SHALLOW TRUE
)
FetchContent_MakeAvailable(googlebenchmark)

ADD_EXECUTABLE(benchmarks ${BENCHMARKS})
TARGET_LINK_LIBRARIES(benchmarks searchlib
  benchmark::benchmark benchmark::benchmark_main)
```

1

FetchContent

Clona a tag v1.9.5 e compila como subprojeto.

2

benchmark_main

Já fornece o main(); o arquivo só tem os benchmarks.

3

Sempre em Release

Medir -O0 não diz nada sobre produção.

Alternativas:

apt install libbenchmark-dev · vcpkg · conan

Anatomia de um benchmark

```
static void BM_LinearSearch(benchmark::State& state)
{
    const auto values = MakeSortedEvens(state.range(0));
    const auto queries = MakeQueries(state.range(0), kN);

    std::size_t i = 0;
    for (auto _ : state)
    {
        bool found = LinearContains(values, queries[i]);
        benchmark::DoNotOptimize(found);
        i = (i + 1) % queries.size();
    }

    state.SetItemsProcessed(state.iterations());
    state.SetComplexityN(state.range(0));
}

BENCHMARK(BM_LinearSearch)
->RangeMultiplier(4)->Range(16, 1 << 16)
->Complexity();
```

1

Setup fora do laço

Montar os dados não entra na medição.

2

for (auto _ : state)

Só o corpo é cronometrado. A lib escolhe quantas vezes rodar.

3

DoNotOptimize

Obriga o compilador a produzir o valor, então ele não pode apagar a chamada.

4

Counters e Big-O

Vazão (items/s) e o N usado no ajuste de complexidade.

5

Registro

N = 16, 64, ..., 65.536, e no final o Big-O ajustado.

Lendo a saída

Execução real no Ryzen 5 5600X, filtrada para a busca linear (cabeçalho resumido)

```
Run on (12 X 4245.21 MHz CPU s)      CPU Caches: L1 32 KiB · L2 512 KiB · L3 32 MiB
-----
Benchmark                          Time          CPU    Iterations UserCounters...
-----
BM_LinearSearch/16                 8.29 ns       8.29 ns    84806745 items_per_second=120.589M/s
BM_LinearSearch/1024               187 ns       186 ns    3917884 items_per_second=5.36194M/s
BM_LinearSearch/65536             10877 ns     10876 ns     61383 items_per_second=91.9456k/s
BM_LinearSearch_BigO               0.17 N        0.17 N
BM_LinearSearch_RMS                1 %           1 %
```

Time

Tempo de relógio por iteração.

CPU

Tempo de CPU da thread.
Se difere de Time, houve espera.

Iterations

Escolhido automaticamente:
84 mi para N=16, 61 mil para N=65.536.

BigO / RMS

Coeficiente ajustado ($0,17 \cdot N$)
e o erro do ajuste.

Parametrizando

Três formas de reaproveitar um benchmark

Argumentos

```
->Arg(1024)
->Args({1024, 8})
->Range(16, 1 << 16)
->RangeMultiplier(4)
->DenseRange(0, 64, 8)

// no corpo:
state.range(0)
state.range(1)
```

Templates

```
template <typename C>
static void BM_Accumulate(
    benchmark::State& state)
{ ... }

BENCHMARK_TEMPLATE(BM_Accumulate,
    std::vector<int32_t>);
BENCHMARK_TEMPLATE(BM_Accumulate,
    std::list<int32_t>);
```

Fixtures

```
class HashFix
    : public benchmark::Fixture {
    void SetUp(const State&)
        override { ... }
};

BENCHMARK_DEFINE_F(HashFix,
    BM_Lookup)(State& st)
{ ... }
BENCHMARK_REGISTER_F(HashFix,
    BM_Lookup)->Range(16, 1<<16);
```

Medindo além do tempo

Complexidade

```
->Complexity()    ou    ->Complexity(benchmark::oNLogN)
```

Com `state.SetComplexityN(n)`, gera as linhas `_BigO` e `_RMS`.

Vazão

```
state.SetItemsProcessed(...)    state.SetBytesProcessed(...)
```

Vira `items_per_second` e `bytes_per_second` na saída.

Counters próprios

```
state.counters["hit_rate"] = benchmark::Counter(v);
```

Qualquer métrica do domínio, com flags de taxa e média.

Controle do timer

```
state.PauseTiming(); /* prepara */ state.ResumeTiming();
```

Tira preparo da medição, mas tem custo próprio: use com cuidado.

Threads e unidade

```
->Threads(4)    ->UseRealTime()    ->Unit(benchmark::kMicrosecond)
```

Mede código concorrente e ajusta a unidade exibida.

Linha de comando e ambiente

<code>--benchmark_filter=Search</code>	regex: roda só o que casar
<code>--benchmark_repetitions=10</code>	repete tudo, gera mean/median/stddev/cv
<code>--benchmark_report_aggregates_only</code>	mostra só os agregados
<code>--benchmark_min_time=2s</code>	tempo mínimo por benchmark (ou 1000x)
<code>--benchmark_min_warmup_time=0.5</code>	aquece antes de medir
<code>--benchmark_out=run.json</code>	salva JSON/CSV para comparar depois
<code>--benchmark_list_tests</code>	lista os benchmarks registrados

Ambiente de medição

- Build Release (-O3)
- Fixar core: taskset -c 2
- Governor em performance
- Máquina ociosa: nada de compilar ou abrir o navegador enquanto mede
- Olhar mediana e CV, não uma execução

PILOTO

Busca em vetor ordenado

Uma biblioteca pequena, quatro implementações e três pegadinhas

4

implementações
de busca

7

tamanhos
16 → 65.536

4,4×

ganho da versão
branchless

O projeto

GBENCH_PILOT: baseado no CPP_SKELETON, com doctest para correção e Google Benchmark para desempenho

```
GBENCH_PILOT/  
├── include/search.h  
├── src/search.cc  
├── test/  
│   ├── unit/main_doctest.cc  
│   └── benchmarks/  
│       ├── bm_search.cc  
│       └── bm_pitfalls.cc  
├── results/*.json  
└── run          # ./run -B
```



LinearContains

Varre o vetor inteiro

$O(N)$



BinaryContains

Binária clássica com if

$O(\log N)$



BranchlessBinaryContains

Binária sem desvio (cmov)

$O(\log N)$



std::unordered_set

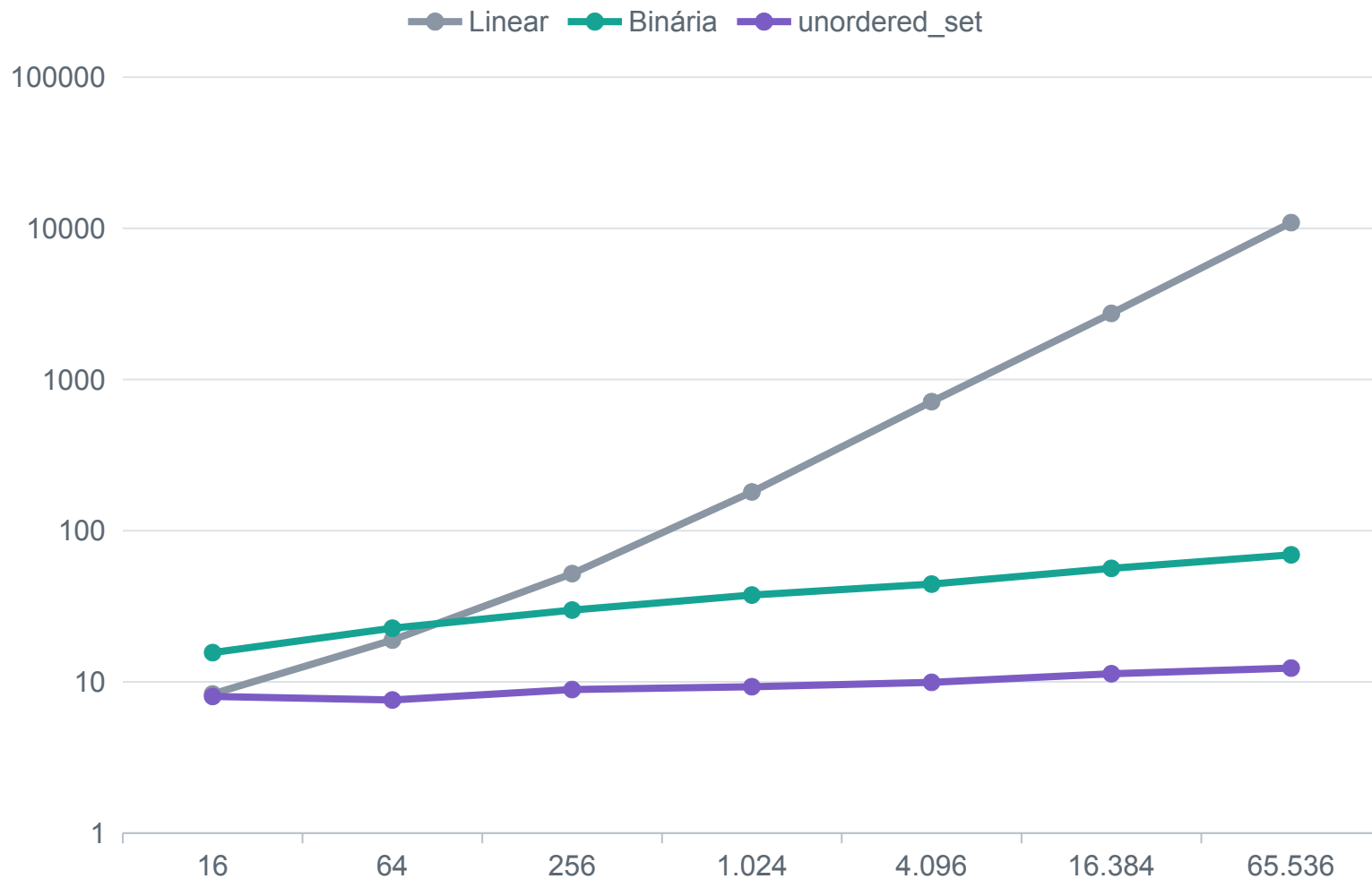
Hash, via fixture

$O(1)$

Ambiente: Ryzen 5 5600X · g++ 15.2 -O3 · taskset -c 2 · governor performance · 5 a 10 repetições, mediana · 65.536 consultas aleatórias

A complexidade aparece nos números

Tempo por consulta (ns, escala log) × tamanho do vetor



Big-O que a lib ajustou

Linear	$0.17 \cdot N$	2%
Binária	$4.00 \cdot \lg N$	7%
Hash	$9.62 \cdot (1)$	17%

Constantes importam. Em $N=16$ a linear (8,3 ns) vence a binária (15,6 ns): varrer 16 inteiros contíguos é barato.

Pegadinha 1: o compilador apagou o benchmark

Mesmo laço, soma de i^2 para $i < 65.536$

SEM DoNotOptimize

```
for (auto _ : state) {  
    int64_t sum = 0;  
    for (int64_t i = 0; i < n; ++i)  
        sum += i * i;           // nunca usado  
}
```

0,000 ns

sem DoNotOptimize: -O3 viu que sum não é usado e removeu o laço

COM DoNotOptimize

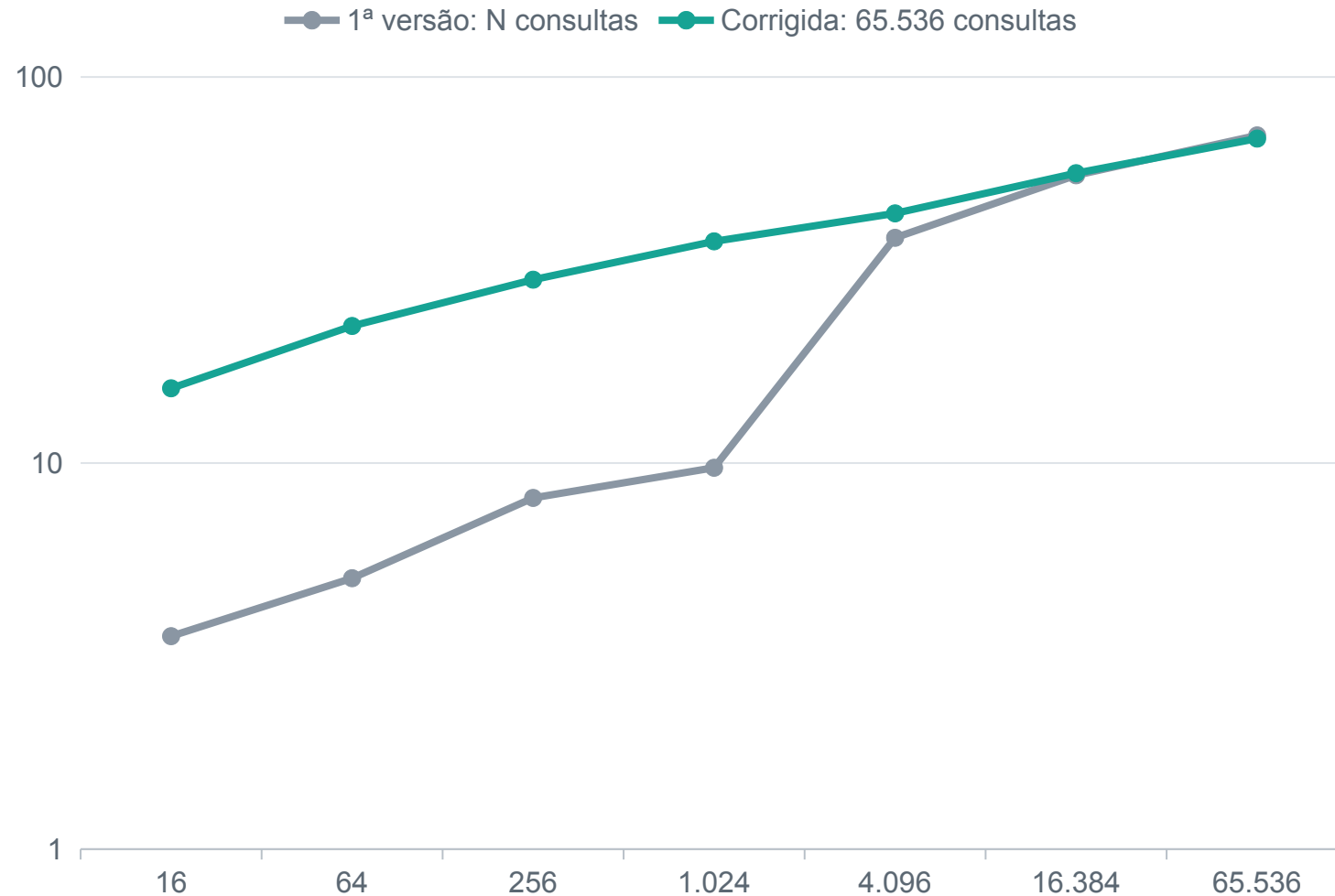
```
for (auto _ : state) {  
    int64_t sum = 0;  
    for (int64_t i = 0; i < n; ++i) {  
        sum += i * i;  
        benchmark::DoNotOptimize(sum);  
    }  
}
```

24,3 μ s

com DoNotOptimize: o trabalho real, 65.536 multiplicações e somas

Pegadinha 2: o hardware decorou o teste

Busca binária, 1ª versão (N consultas em ciclo) × versão corrigida (65.536 consultas)



3,9× otimista

em N=1.024: 9,7 ns medidos, 37,5 ns reais

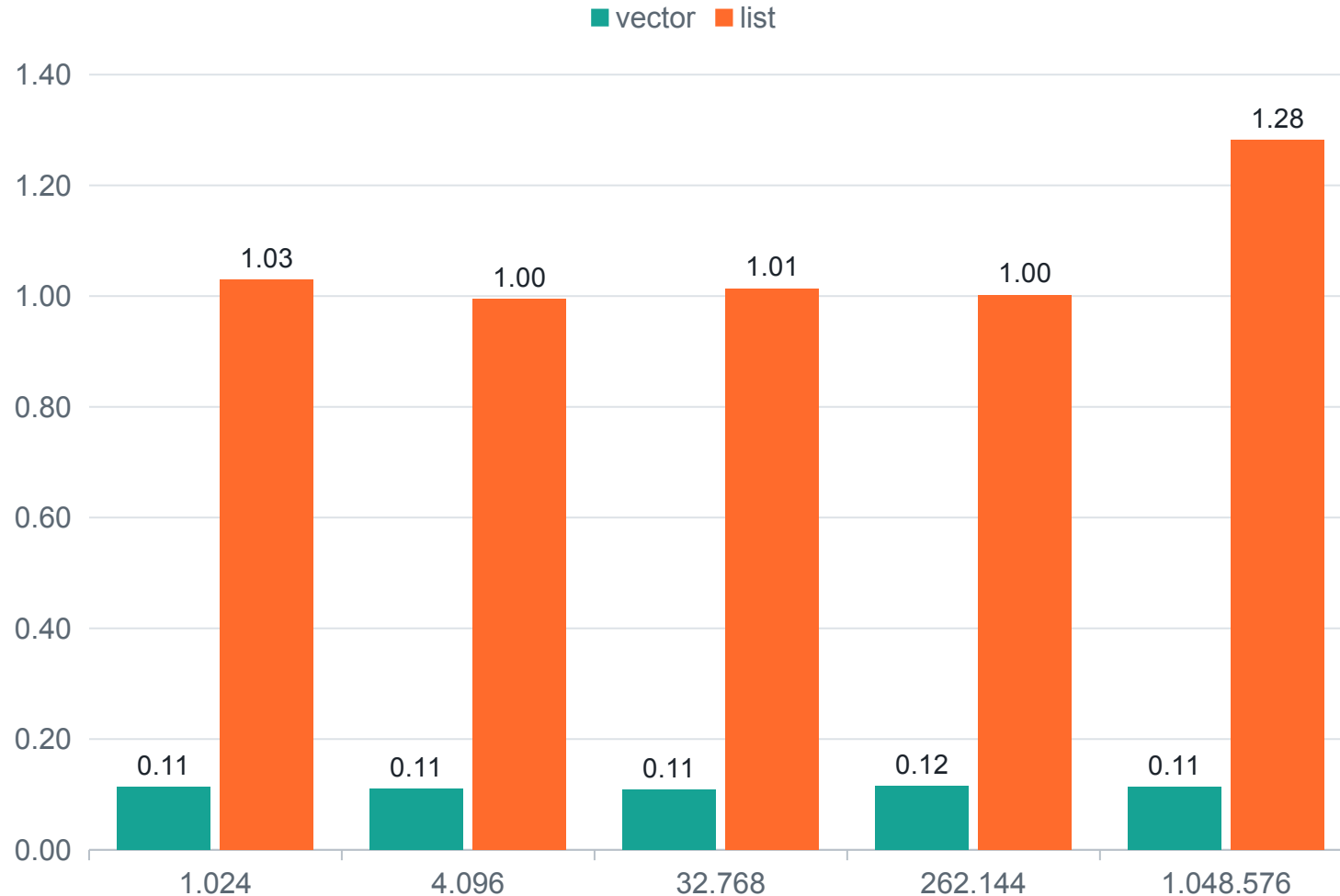
RMS 55% → 7%

erro do ajuste $O(\log N)$

Lição: com poucas consultas repetidas, o preditor de desvios da CPU aprende o padrão. Big-O que não encaixa é sinal de benchmark errado.

Pegadinha 3: mesmo $O(N)$, custo 10× diferente

std::accumulate em vector × list: ns por elemento (BENCHMARK_TEMPLATE)



37 GB/s

vector: memória contígua, prefetch e SIMD

3,9 GB/s

list: cada nó é um ponteiro para outro lugar da memória

Lição: a complexidade é igual; cache e layout de memória decidem o resultado.

Otimização guiada por benchmark

Hipótese: a binária perde tempo com desvios mal previstos. Solução: tirar o if do laço.

```
// antes: um if difícil de prever
while (lo < hi) {
    mid = lo + (hi - lo) / 2;
    if (values[mid] < target) lo = mid + 1;
    else                      hi = mid;
}

// depois: seleção sem desvio
while (len > 1) {
    half = len / 2;
    base = (base[half] < target)
           ? base + half : base;
    len -= half;
}
```

CONFERINDO O ASSEMBLY

```
$ objdump -d bin/Release/benchmarks
cmp    (%rsi),%edi
cmovg  %rsi,%rdx    <- sem jump
sub    %rcx,%rax
```

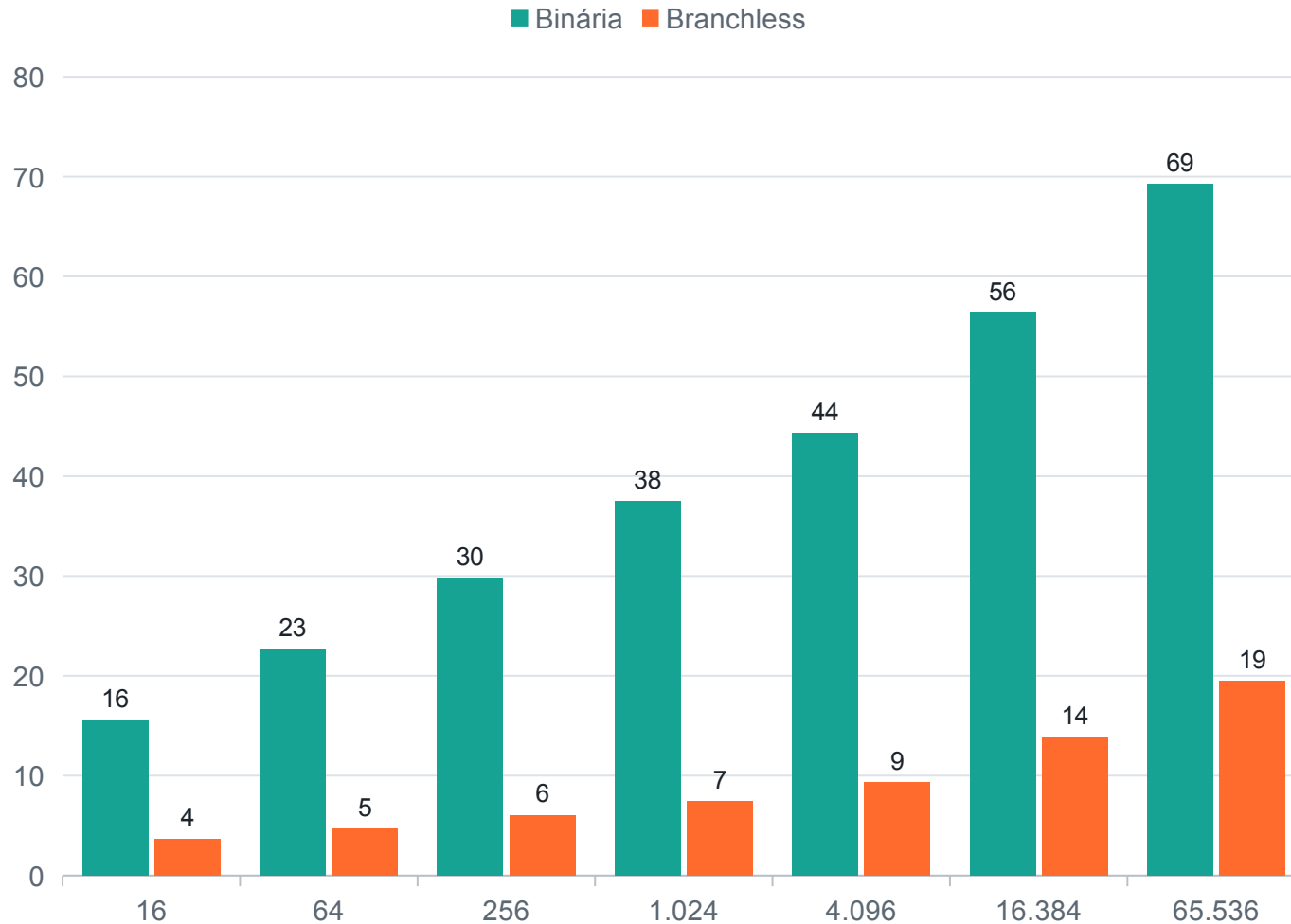
O laço executa sempre o mesmo número de passos; só o endereço muda.

A CPU não precisa adivinhar o lado da busca, e cada erro de previsão custa da ordem de 15 ciclos.

Os testes unitários confirmam que o resultado é igual ao da linear.

Resultado: compare.py

Tempo por consulta (ns), mediana de 10 repetições



-77%

tempo (média geométrica em todos os N)

p = 0,0002

Mann-Whitney U-test, 10 × 10 repetições

```
$ compare.py benchmarksfiltered \  
    v2.json BM_BinarySearch \  
    v2.json BM_BranchlessBinarySearch  
OVERALL_GEOMEAN    -0.7748
```

Checklist de boas práticas



Release, flags de produção

-O3, as mesmas flags do binário real



DoNotOptimize no resultado

E ClobberMemory quando escreve em memória



Repetições + mediana + CV

--benchmark_repetitions, CV abaixo de 2%



Compare com estatística

compare.py e p-valor, não comparar no olho



Só o essencial no laço

Setup fora; PauseTiming só se não houver alternativa



Dados realistas e variados

Padrões repetidos são aprendidos pela CPU



Ambiente controlado

taskset, governor performance, máquina ociosa



Microbenchmark ≠ sistema

Confirme no programa real com perf ou um profiler

Obrigado!

Perguntas?

Repositório

`github.com/google/benchmark`

Guia do usuário

`github.com/google/benchmark/blob/main/docs/user_guide.md`

Comparação A/B

`github.com/google/benchmark/blob/main/docs/tools.md`

Código do piloto

`GBENCH_PILOT (./run -B)`